

8.1 Objective Functions and Empirical Risk

Supervised learning algorithms formulate training as minimizing an empirical risk loss function $L(\theta)$ evaluated over a finite dataset of training examples.

The loss function measures the discrepancy between model predictions and true targets, common examples being mean squared error for regression and cross-entropy loss for classification tasks.

Because deep neural networks are non-linear parameterizations, the loss surface is non-convex, characterized by saddle points, ravines, and local minima in million-dimensional parameter spaces.

8.2 Gradient Descent and Learning Rates

Gradient descent minimizes the objective function by taking iterative steps in the direction of steepest descent, defined by the negative gradient vector $-\text{grad } L(\theta)$.

The parameter update equation is $\theta_{t+1} = \theta_t - \eta \text{grad } L(\theta_t)$, where η is the learning rate hyperparameter governing step magnitude.

Selecting an optimal learning rate is critical: an excessively large learning rate causes divergence and numerical instability, whereas an overly small learning rate leads to glacial convergence.

8.3 Stochastic Gradient Descent and Adam Optimizer

Stochastic Gradient Descent (SGD) computes the gradient over a mini-batch of examples rather than the entire dataset, drastically reducing computational overhead per update and introducing helpful regularization noise.

To accelerate convergence in ill-conditioned ravines, momentum methods accumulate an exponentially decaying moving average of past gradients to maintain directional velocity.

The Adam optimizer combines momentum with adaptive learning rates by maintaining estimates of both first and second raw moments of the gradients, scaling updates inversely with gradient variance.